Parsek Roadmap

Philosophy

Parsek is a **git-like recording system** for KSP missions. You record flights sequentially, commit them to a timeline, and they play back automatically as ghost vessels while you fly new missions. There is one timeline, recordings are immutable commits, and the game always moves forward.

Like git, you can go back to any earlier point and start new work. Existing recordings don't change — they play out as ghosts alongside your new missions. There is no branching, no state reversal, and no paradoxes. Conflicts are prevented through resource budgeting: committed recordings have already claimed their costs, so the player can only spend what's actually available.

Time travel paradoxes are avoided by two invariants: **causality** (events are always processed in time-axis order) and **additivity** (the timeline is append-only — committed recordings and game actions can be added but never deleted). The world state at any point in time is fully determined by the ordered sequence of committed events before it. When a new event is inserted into the timeline (e.g., a recording committed at an earlier UT after a rewind), all derived resource values — available science, funds, reputation — are recalculated from scratch across the full timeline. Events themselves are immutable; the computed state that flows from them is not stored but recomputed whenever the event set changes.

Current focus: release readiness (owner ruling, 2026-10-24)

The work now is bringing Parsek to a quality, bug-free state fit for a public release: fixing open defects, live-proving fixes that so far have only unit tests, and closing the gaps the harness finds. Owner ruling (V, 2026-10-24): "we're not starting work on the multiplayer implementation phase; we have to bring Parsek to a sufficient quality bug-free state so that we can release it publicly first; multiplayer comes later, if we have users". Phase 14 is therefore deferred until after the public release, and only if Parsek has users. The co-op design stays as written and is not being implemented.

Completed

v0.3 — Foundation

Initial release. The core loop shipped end-to-end: record a flight, commit it, rewind, see it play back as a ghost alongside new missions.

Recording & playback: Position recording with adaptive sampling and geographic coordinates. Kinematic ghost playback with opaque vessel replicas. Orbital recording with analytical Keplerian orbits. Auto-recording on launch and EVA. Chained recordings across vessel lifecycle events (undock, EVA, dock). Context-aware merge dialog (Merge to Timeline / Discard). Vessel persistence with deferred spawn, crew reservation, and resource deltas.

Ghost visual fidelity: Comprehensive part event types recorded and replayed on ghost vessels — decoupling, staging, parachutes, engines (with FX), solar panels, antennas, radiators, landing gear, lights, cargo bays, fairings, RCS (with FX), docking/undocking, inventory parts.

Career mode: Milestones (tech research, part purchases, facility upgrades, contracts, crew changes) captured as immutable timeline commits. Resource budget computed on-the-fly from recordings + milestones. Epoch isolation to prevent abandoned timeline branches from leaking. Resource deduction on revert so KSP's top bar reflects available resources. Action blocking to prevent re-researching committed tech or re-upgrading committed facilities. Per-recording rewind saves with quicksave-based timeline restoration. Rewind UI with confirmation dialog, resource reset, crew re-reservation, and action replay. Fast-forward to advance past committed recordings.

Recordings Manager UI with sortable columns, per-recording loop/hide, rewind/fast-forward, and status indicators.

Design: `docs/dev/done/design-restore-points.md` , `docs/dev/done/design-going-back-in-time.md`

v0.4 — Visual Fidelity & Polish

Orbital rotation fidelity, watch camera, heat effects, KSC scene playback, UI polish, and code refactoring.

Orbital rotation: Ghost vessels in orbital segments preserve their recorded attitude instead of always facing prograde. Rotation stored relative to the orbital velocity frame at the on-rails boundary and reconstructed from the Keplerian orbit. SAS-locked orientations hold correctly throughout the orbit. PersistentRotation mod detected and supported — spinning vessels reproduced during playback.

Watch camera: Camera follows a ghost vessel during playback. Automatically recenters on visible parts after separation events.

Ghost visuals: Re-entry heating FX with mesh-surface fire particles. Heat shield ablation, smoke and spark FX on decouple/destroy. Fairing ghost visuals. EVA kerbal facing fix. Explosion visual effect on impact with camera hold.

KSC playback: Ghosts visible in KSC scene with overlap support and distance-based part event culling.

UI & usability: Per-save settings panel (auto-record, sampling thresholds). Recording stats (max altitude, max speed, max G, distance, duration). Non-revert "Commit Flight" button. Two-phase parachute deploy. Auto-loop with per-recording interval controls. Recording groups with multi-membership. Time warp visual cutoffs and deferred spawn queue. Context-aware rewind button.

Code health: Major refactor reducing coupling between modules. Sandbox rewind fix.

Design: `docs/dev/done/design-orbital-rotation.md`, `docs/dev/done/design-camera-follow-ghost.md`

v0.5 — Recording System Redesign

Comprehensive redesign of the recording and playback systems. Multi-vessel sessions, ghost chain paradox prevention, spawn safety, time jump, rendering zones, and ghost visual hardening.

Design: `docs/parsek-flight-recorder-design.md` (consolidated design document)

Recording model:

- Segment boundary rule — only physical structural separation creates new segments; controller changes, part destruction without splitting, and crew transfers are SegmentEvents within a continuing segment
- Crash coalescing — rapid split events grouped into single BREAKUP events
- Environment taxonomy — classification (Atmospheric, ExoPropulsive, ExoBallistic, SurfaceMobile, SurfaceStationary) with hysteresis
- TrackSections — typed trajectory chunks tagged with environment, reference frame, and data source
- Reference frames — ABSOLUTE for physics, ORBITAL_CHECKPOINT for on-rails, RELATIVE for anchor-vessel proximity

Multi-vessel sessions:

- Recording tree (DAG) — multi-vessel missions recorded as a single unit with split/merge tracking across undock, EVA, dock, joint break, and breakup events
- Background vessel recording — all vessels in the physics bubble sampled at proximity-based rates with full part event capture
- Booster/debris recording — automatic tree promotion on staging with debris TTL
- Highest-fidelity-wins merge — overlapping Active/Background/Checkpoint data merged per vessel
- Flag planting recording and playback

Ghost chain paradox prevention:

- Committed recordings that interact with pre-existing vessels cause those vessels to become ghosts from rewind until the chain tip resolves
- Intermediate spawn suppression — multi-link chains only spawn at the tip
- Ghost conversion — real vessels despawned and replaced with ghost GameObjects during chain windows
- PID preservation — chain-tip spawns preserve the original vessel's persistent ID for cross-tree chain linking
- Ghosting trigger taxonomy — structural events, orbital changes, and part state changes trigger ghosting; cosmetic events do not

Spawn safety:

- Bounding box collision detection — spawn blocked when overlapping with loaded vessels
- Ghost extension — ghost continues on propagated orbit/surface past recording end while spawn is blocked
- Trajectory walkback — for immovable blockers, walks backward along recorded trajectory to find a valid spawn position
- Terrain correction — surface spawns adjusted for terrain height changes between recording and playback
- KSC exclusion zone — 50m safety radius around launch pad and runway

- Spawn-die-respawn prevention (3-cycle limit), spawn abandon flags, retry limits (150 frames)
- Snapshot situation correction (FLYING→LANDED for landed terminals, FLYING→ORBITING for orbital terminals)
- Non-leaf tree recording spawn suppression, unsafe snapshot situation blocking
- Dead crew protection on spawn (strip dead crew, abandon all-dead spawn)

Time jump:

- Relative-state time jump — discrete UT skip preserving rendezvous geometry across ghost chain windows
- Epoch-shifted orbits — orbital elements recomputed at the new UT from state vectors

Rendering & performance:

- Distance-based zones — Physics, Visual, Beyond — with zone-aware playback and part event gating
- Ghost soft caps — configurable thresholds with priority-based despawning, reduced fidelity (75% renderer culling), simplified orbit-line mode (removed in 0.8.1, PR #226: replaced by the distance-based ghost LOD)
- Terminated chain early-out — fully-terminated trees and chains skip per-frame evaluation

Ghost visual hardening: Variant textures and materials (TEXTURE/MATERIAL/GAMEOBJECT rules), damaged wheel filtering, fairing meshes with procedural truss and nosecone caps, SRB nozzle glow (FXModuleAnimateThrottle), engine shrouds with variant awareness, initial state seeding for all 16 tracking set types. Compound part visuals (fuel lines, struts). Plume and smoke trail fixes (KSPParticleEmitter native control, emission module disabled). Control surfaces, robotics/servo detection, cabin lights, animation-based deployables. RCS debounce (8-frame threshold). Part separation smoke/spark FX. Lingering particle systems on ghost despawn.

Ghost world presence: CommNet relay: only the antenna-power math exists; it was never wired into the game (todo GUI-D3-GHOSTCOMMNETRELAY-IS-DEAD-WHILE-A-LIVE-PATCH-CITES-IT-AS-JUSTIFICATION). Ghost labels. ProtoVessel-based map presence — tracking station entries, orbit lines, navigation targeting, ghost icon click popup (Set Target / Watch / Focus). Harmony guard rails (27 checks across 9 files, 4+ patches). Orbit segment updates on SOI transitions. Tracking station NRE protection.

Real Spawn Control: Proximity-based UI for warping to when nearby ghost craft become real vessels. Per-craft warp buttons, sortable columns, countdown display, screen notifications. Countdown column in Recordings Manager.

Per-phase looping: Chain recordings split at atmosphere/altitude/SOI boundaries. Tree recordings split by optimizer at environment boundaries. Auto loop range trims boring bookends. Debris loop sync via LoopSyncParentIdx.

Watch mode hardening: Zone-hide exemption for watched ghosts (no 120km cutoff). Hold timer with retry for chain transitions. Auto-follow through tree branching with PID-matched descent. FF watch transfer. Camera bridge anchors at cycle boundaries. Distance guard (100km rendering-safe limit). Orbital recording exemption from distance cutoffs.

Rewind/spawn hardening: Future vessel strip (PRELAUNCH and all types via quicksave PID whitelist). Spawn state reconciliation after strip. EVA vessel removal with crew rescue. Localization key resolution at all comparison sites. Spawn position from snapshot (not trajectory endpoint). EVA endpoint spawning. Ladder state stripping. Orbital vessel spawn via state vector orbit construction.

UI: Fast-forward redesigned as instant UT jump. Simplified merge dialog. Compressed Status column (merged countdown). Group header aggregate stats. Recording group unique naming. In-game test runner (90 tests, Ctrl+Shift+T).

Code health: GhostPlaybackEngine extraction (zero Recording references, IPlaybackTrajectory interface). ChainSegmentManager extraction. ParsekPlaybackPolicy event-driven architecture. Comprehensive log audit (92% output reduction). 4600+ xUnit tests. PartStateSeeder unification. GhostBuildResult consolidation.

v0.6 — Game Actions System

Full redesign of milestone capture, resource budgeting, and action replay. Standalone resource ledger tracking every economic event on the timeline.

Design: `docs/parsek-game-actions-and-resources-recorder-design.md`

Architecture: Sidecar files capture raw data during flight; extraction to ledger on commit. Two-phase recalculate+patch on warp exit. Unified recalculation walk from UT=0 with two-tier module dependency ordering. KSP UI patched to display available funds (not gross balance).

Resource modules (8):

Module	Key mechanics
Science	Immutable awarded values + derived effective values, per-subject caps via full recalculation sorted by UT
Funds	Seeded balance from save, reservation system prevents overspending across rewinds, vessel build costs as recording-associated spendings
Reputation	Non-linear gain/loss curve, nominal vs effective values
Milestones	Once-ever binary cap, chronological priority, first-tier feeds funds and reputation
Kerbals	IResourceModule participant, reservation from UT=0 as continuous block, temporary vs permanent loss, replacement chains, stand-in generation, managed-kerbal filtering
Facilities	Upgrade/destruction/repair schemas, visual state management during fast-forward, KSP state patching, deferred replay for missing facilityRefs
Contracts	UT=0 reservation, accept/complete/fail/cancel lifecycle, parameter progress tracking, deadline failure detection
Strategies	UT=0 reservation, transform layer between first-tier and second-tier modules, commitment rates

Paradox prevention: No-delete invariant, spending reservation (science/funds), UT=0 reservation (kerbals/contracts/strategies). Milestone path qualification. Budget deduction clamping. Chain gap closure for game state events.

Crew lifecycle: Crew reservation via internal managed state (Harmony patches for KSP roster validation and astronaut complex count). Crew swap in both flight and KSC spawn snapshots. Orphaned crew rescue after vessel strip. Dead crew spawn protection. Kerbal rescue detection. Managed-kerbal event filtering.

Validated across all game modes: Sandbox (no resources), Science (science-only), Career (full complexity). 4621 tests passing.

v0.7 — Timeline

Unified chronological view of all committed career events, replacing the Game Actions window.

Design: `docs/parsek-timeline-design.md`

Timeline window: Flat chronological list of career events sorted by UT. Current-UT divider separates past (full color) from future (dimmed). Two significance tiers: Overview (mission structure — launches, milestones, contracts, facilities) and Detail (all resource transactions). Three source filters: Recordings, Actions (deliberate player choices), Events (gameplay consequences). Vessel-level telemetry (part events, segment events) stays in the Recordings Manager.

Entry display: EVA-aware (`EVA: Jeb from Mun Lander (MET 5s)` , `Board: Jeb (Mun Lander)`). Spawn at EndUT with VesselSituation (`Spawn: Vessel (Landed on Mun)`). Mission Elapsed Time on launches (KSP calendar). Humanized science subjects, tech nodes, milestones, strategy names. Color-coded: green = earnings, red = penalties, light blue = player actions, white = recordings.

Operations: Rewind (R) and Fast-Forward (FF) buttons on recording entries. GoTo cross-link opens Recordings Manager, unhides recording, expands parent groups, scrolls to target.

Data model: 26 entry types (2 recording lifecycle + 23 game actions + 1 legacy). `TimelineBuilder` with 3 collectors + stable UT sort. `IsPlayerAction` classification. Game-mode aware (career/science/sandbox). 4870 tests passing (53 timeline-specific).

Location context: Recordings become location-aware — body, biome, situation, and stock launch site name captured at recording start and end. Timeline shows location-enriched entries ("Launch: Vessel from Launch Pad on Kerbin", "Spawn: Vessel (Landed at Midlands on Mun)"). Sortable Site column in Recordings Manager with UT tiebreak. Prerequisite for logistics routes and async multiplayer.

v0.8 — Resource Snapshots & Recording Optimization

The logistics-prerequisite release line. Phase 11 and Phase 11.5 shipped across `v0.8.x` , adding resource/inventory/crew manifests to recordings and the observability/optimization work needed before long-lived looped transport routes.

Phase 11 — Resource snapshots: recordings now capture start/end resource manifests, inventory manifests, crew manifests, and dock-target identity at the boundaries needed for future route endpoint and delivery logic.

Phase 11.5 — recording optimization & observability: storage/perf diagnostics are now exposed in-game, Flight ghost LOD is live, and compact trajectory/snapshot sidecars plus readable mirrors cut comparable recording payload size by `85.17%`.

Phase 11: Resource Snapshots

Recordings capture physical resource manifests at recording start and end.

Resource manifests (implemented): `ExtractResourceManifest` walks vessel snapshot PART > RESOURCE nodes, sums by resource name. `StartResources` / `EndResources` fields on Recording, serialized as additive RESOURCE_MANIFEST ConfigNode. Captured at 8 boundary sites (recording start/stop, chain boundaries, breakup, background splits, optimizer splits/merges). EC and IntakeAir excluded (environmental noise). Dock target vessel PID (`DockTargetVesselPid`) captured at dock boundaries for route endpoint identification. Hover tooltip in Recordings Manager shows per-resource start-to-end with delta.

Inventory manifests (implemented): KSP 1.12 `ModuleInventoryPart` items (stored parts in cargo containers). `ExtractInventoryManifest` walks MODULE > STOREDPARTS > STOREDPART nodes. `InventoryItem { count, slotsTaken }` struct + vessel-level `totalInventorySlots`. Same capture sites as resources.

Crew manifests (implemented): Crew composition by trait (Pilot/Scientist/Engineer/Tourist). Route delivery uses generic kerbals (separate from crew reservation system). Same capture pattern.

Phase 11.5: Recording Optimization & Observability

Optimization pass before logistics routes add many long-lived looped recordings. A long career with dozens of missions will accumulate significant disk, memory, and playback pressure. Phase 11.5 now has three shipped threads and one explicit follow-up:

1. observability and diagnostics so storage/perf pressure is measurable during playtests
2. playback-side optimization via the current Flight ghost LOD policy
3. recording-side optimization across both trajectory and snapshot sidecars
4. remaining follow-up: synthetic stress benchmarking/tuning for the shipped policy and formats

Observability (shipped)

Phase 11.5 shipped the measurement/reporting pieces needed to make optimization work evidence-based:

- **Per-save storage report** — total disk size of Parsek data (sidecar files + `.sfs` metadata), broken down by recording in diagnostics.
- **Per-recording stats** — point count, part event count, orbit segment count, sidecar file sizes (`.prec` , `_vessel.craft` , `_ghost.craft`) visible in diagnostics / recording details.
- **Playback budget visibility** — playback timings, zone behavior, and FX counts exposed through diagnostics and one-shot/rate-limited logging.
- **Memory / LOD visibility** — live diagnostics for active ghost buckets and hidden-tier shell state, plus spawn/destroy timings for recent ghost lifecycle work.

Flight Playback / LOD (shipped)

Phase 11.5 also shipped the current Flight ghost LOD policy used to keep replay density sane during playtests:

- shared internal distance thresholds (`2.3 km` , `50 km` , `120 km` , watch cutoff)
- watched ghosts inside cutoff forced to full fidelity
- unwatched reduced tier (`2.3-50 km`)
- unwatched hidden-mesh tier (`50-120 km`)
- hidden-tier shells keep logical playback alive while unloading built mesh/resources
- live diagnostics reporting for `full / reduced / hidden / watched override`

Recording Storage (shipped)

The storage-focused half of Phase 11.5 removed the biggest measured trajectory-side waste without changing visible playback contracts:

- **Authoritative section sidecars** — `v1` `.prec` files stop duplicating flat `POINT` / `ORBIT_SEGMENT` data when `TrackSections` already contain the same trajectory.

- **Ghost snapshot alias mode** — identical ghost/vessel snapshots are stored once via `ghostSnapshotMode` metadata instead of always writing duplicate `_ghost.craft` files.
- **Compact binary trajectory sidecars** — current-format `.prec` files now use header-dispatched binary `v3` with exact scalar payloads, a file-level string table, and conservative sparse defaults for stable body/career point fields.
- **Compact lossless snapshot sidecars** — `_vessel.craft` / `_ghost.craft` now write as lossless `Deflate` envelopes at the highest built-in .NET compression level while still loading legacy text snapshot files.
- **Readable mirror sidecars** — default-on `.prec.txt`, `_vessel.craft.txt`, and `_ghost.craft.txt` mirrors keep the compact binary/lossless files authoritative while preserving a human-readable debugging path and diagnostics byte accounting.
- **Storage regression harness** — representative fixtures, mixed-format round-trips, sidecar log assertions, and scenario-writer coverage protect the new format path.

Measured outcome so far: recording sidecars are no longer dominated by text storage. In the April 13, 2026 comparable log-bundle corpus, authoritative `.prec`, `_vessel.craft`, and `_ghost.craft` files totaled `1.34 MB` versus `9.03 MB` for readable text mirrors (`7.69 MB` saved, `85.17%` smaller), and the latest bundle alone dropped from `745,079 B` to `102,845 B` (`86.20%` smaller).

Remaining Follow-Up

With trajectory-side and snapshot-side storage optimization now shipped, the remaining Phase 11.5 follow-up is validation/tuning work against larger corpora rather than another planned format pass:

- run synthetic stress benchmarking against the shipped ghost LOD/storage stack
- tune thresholds or hot spots only if diagnostics show real pressure at scale
- keep future storage changes measurement-driven instead of pre-committing to another rewrite
- defer trajectory thinning, compression, or lazy loading until the snapshot bucket is re-measured and still justifies more work

v0.9 — Rewind to Separation

Phase 12 shipped: re-fly unfinished missions after multi-controllable splits (staging, undock, EVA with 2+ controllable outputs). Owner ruling 2026-09-27 narrowed it to vessel separations only (staging, decoupling, undocking): an EVA never gets a Rewind Point or a Re-Fly. Full design: `docs/parsek-rewind-to-separation-design.md`. The pre-implementation spec that drove v0.9 is archived at `docs/dev/done/parsek-rewind-separation-design.md`.

- **Rewind Points at split time** — every multi-controllable split writes a KSP quicksave to `saves/<save>/Parsek/RewindPoints/<rpId>.sfs` plus a persistent-id-to-slot map captured at save time, so the post-load strip can reliably identify which vessels to replace with ghosts.
- **Unfinished Flights UI group** — a virtual, computed group in the Recordings Manager lists every committed BG-crash sibling whose parent split has a Rewind Point. Non-hideable, non-drop-target; clicking Rewind re-flies the sibling from the moment of the split.
- **Append-only supersede** — merging a re-fly appends a `RecordingSupersede(old, new)` relation. The original recording is never mutated or deleted; ghost/claim subsystems filter via the relation list. Tree invariant preserved (additive only).
- **Reviewed supersede tombstone scope** — merge tombstones can retire recording-scoped career actions from the superseded subtree, including contracts, milestones, facilities, strategies, science, funds/reputation, and kerbal consequences. Seeds, null-scoped KSC/system rows, already-paid rollout costs, and unknown future action types stay preserved until reviewed.
- **Crashed re-fly stays rewindable** — merging a crashed attempt commits as `CommittedProvisional` and remains an Unfinished Flight the player can try again. Merging a stable outcome seals the slot as `Immutable`.
- **Effective-state model** — `EffectiveRecordingSet` and `EffectiveLedgerSet` give every subsystem one rule for "what counts right now." A narrow session-suppressed subtree carve-out applies during an active re-fly only to physical-visibility subsystems (ghost walker, claim tracker, map / tracking-station / CommNet). Career state reads ERS/ELS directly.
- **Journaled staged merge** — irreversible file operations (deleting reap-eligible Rewind Point quicksaves) happen only after a durable save, and the nine-phase `MergeJournal` recovers the merge on the next load if a crash occurs mid-sequence.
- **Revert-during-re-fly dialog** — intercepts stock Revert-to-Launch while a session is active and offers Retry from Rewind Point / Discard Re-fly / Continue Flying.

---

v0.9.1 — Unfinished Flights Stable Leaves & Re-Fly Hardening

The 0.9.1 line shipped the Phase 12.5 Unfinished Flights stable-leaf extension and a focused Re-Fly hardening pass. Full design (now folded into the unified Rewind-to-Separation spec): `docs/parsek-rewind-to-separation-design.md`. Pre-implementation spec for the stable-leaf extension is archived at `docs/dev/done/parsek-unfinished-flights-stable-leaves-design.md`; research note (R17) at `docs/dev/research/extending-rewind-to-stable-leaves.md`.

- **Broader Unfinished Flights predicate** — `IsUnfinishedFlight` now includes controllable non-focus Rewind Point children that end `Orbiting` or `SubOrbital`, plus stranded EVA kerbals with non-boarded terminal states (the EVA branch was removed by the 2026-09-27 ruling: an EVA kerbal is never an Unfinished Flight). Focus-continuation upper stages, debris, and successful auto-recovered boosters stay out of the list.
- **STASH system group and row actions** — the virtual Unfinished Flights group is now displayed as `STASH`; rows expose `Fly` and `Seal`. `Seal` permanently closes a slot by flipping its effective tip recording to `Immutable`, and a stable terminal leaf backed by a Rewind Point can be manually `Stash`ed (which opens it by demoting its tip to `CommittedProvisional`). A slot's open/closed state is read solely from its tip `MergeState`, the single source of truth.
- **Persistent slot signals** — `ChildSlot.Stashed`, `RewindPoint.FocusSlotIndex`, and `ReFlySessionMarker.SupersedeTargetId` preserve manual stash intent, focus, and linear re-fly chain extension across save/load. (Open/closed itself lives in the recording's `MergeState`, not a separate slot bit.)
- **Helper extraction** — Unfinished Flight membership and route resolution now live outside the table UI so commit/merge/reap code can use the same predicate as the Recordings Manager.
- **Re-Fly supersede linearization** — repeated re-flies append from the slot's prior effective tip rather than creating origin-rooted star relations. Legacy star-shaped portions are tolerated while new appends form the dominant linear chain.
- **Re-Fly visibility and resume hardening** — session suppression no longer hides unrelated side-off siblings, raw playback/materialization paths share relation-supersede filtering, doubled GhostMap ProtoVessels are suppressed during pending-tree restore windows, and quickload-resume keeps active-only trim scope intact.
- **Ghost trajectory follow-through** — 0.9.1 also folded in the current ghost trajectory rendering hardening: continuous terrain correction, structural-event snapshots, outlier rejection, anchor/co-bubble fixes, and recorded-anchor fallbacks for Re-Fly relative playback.

v0.10 - Mission Abstractions & Map/TS Render Redesign

The 0.10 cycle shipped two major features on top of the looping system: a mission abstraction layer that loops whole missions as a unit, and a clean modular rewrite of the map-view and Tracking-Station ghost render path. Designs: `docs/dev/design-mission-abstractions.md`, `docs/dev/design-mission-periodicity.md`, `docs/dev/design-map-ts-render-architecture.md` (plus the tracer design and the phased migration plan + status under `docs/dev/plans/`).

- **Mission abstractions and mission loops** - a new Missions tab groups a recording tree into a configurable mission (one continuous vessel per row; branches included or excluded with checkboxes; clonable into differently-configured variants). A mission can be looped as a single unit: the whole mission replays together on one shared clock in flight, the Space Center, and the Tracking Station, instead of each recording looping on its own. Watching a looped mission follows the live vessel and hands the camera to the next stage as the shared clock crosses each boundary.
- **Self-overlapping missions** - a looped mission whose period is shorter than its length now overlaps itself: it relaunches every period (in game-time, so faster under warp) so several staggered replays of the whole mission play at once.
- **Pad-aligned and transfer-window relaunch** - a looped mission relaunches at the right time so its replay lines up with the live sky. A launch-and-orbit mission relaunches when the pad has rotated back under the recorded orbit; a Mun or Minmus mission reschedules each relaunch to a window where the pad lines up and the moon is close enough that the recorded transfer still reaches it. A "Warp to..." button jumps to the next window; an atmosphere-only or surface mission loops continuously.
- **Map / Tracking-Station render redesign** - the ghost map/TS render path was rebuilt as a clean modular pipeline (chain assemble, sample, decide, treatment, draw) that replaces the prior smear of Harmony patches, OnGUI passes, and lifecycle ticks, driving each ghost's orbit line and icon from one source. It fixes a family of long-standing artifacts: looped and re-aimed icons ride their own orbit line, trajectory lines and labels stay put when panning (no jitter or doubled lines), escape and flyby lines clip to the part actually flown, and a self-overlapping looped mission shows one icon or marker per live replay in both the map and the Tracking Station, matching flight.
- **Logistics (Supply Routes) v0** - the Phase 13 logistics feature shipped in 0.10.0. Fly a cargo run once (launch from KSC, dock at a destination, transfer stock resources or inventory, undock) and commit it; a sealed run with one complete dock-deliver-undock window then surfaces as a Candidate in the new Logistics window, where one click turns it into a recurring Supply Route. The route replays the recorded run as a looped mission segment and delivers the same cargo at the dock each cycle, with a UT-driven scheduler, adjustable dispatch cadence, endpoint resolution (orbital by PID, surface with a nearest-vessel fallback), and a route ledger that participates in rewind / re-fly. Career KSC-origin routes charge a stock-realistic per-run funds cost and credit the recovered transport cost back one cycle later. Only KSC-origin routes dispatch in v0: a non-KSC (vessel) origin route can be created (its origin proof is captured) but the per-cycle origin-cargo debit is stubbed, so it holds without dispatching. The v0 cut is deliberately narrow (docking-only, delivery-only, single-stop, same-body, KSC-origin); pickup, mixed pickup/delivery, multi-stop, round-trip, non-docking connections, non-KSC origin cargo debit (docked and undocked), and inter-body re-aimed routes are future work. Full design: `docs/parsek-logistics-supply-routes-design.md`.

v0.10.1 - Re-aim Maturity, Logistics Network, Waterfall / ReStock

Point release maturing the re-aim / looping engine and expanding Supply Routes into a real logistics network.

- **Re-aim reliability** - looped interplanetary missions now re-aim far more launch windows: heliocentric two-burn (parking-orbit) departures, eccentric targets (Eeloo, Moho), near-Hohmann half-orbit angles, and steeply inclined targets all resolve a re-aimed transfer instead of falling back to a faithful replay; windows that still cannot solve fail closed cleanly.
- **Station phase-lock and arrival holds** - missions that rendezvous with a station relaunch phase-locked to the station's live orbit (launch-body, Mun / Minmus, and interplanetary destinations), with per-loop parking-loiter re-timing so nearly every pad window aligns; the station phase tolerance widened to 3 degrees for sustained daily windows. Launch-to-escape seams, derotated rendezvous burns, and looped landings after destination parking all render connected.
- **Logistics network (M1-M4)** - Supply Routes gained non-KSC docked origins (hub-and-spoke via chained routes), any-mod resources plus mining / harvest provenance, pickup and mixed pickup / delivery windows, and multi-stop / multi-origin / round-trip shapes. Dispatch priority, hold-reason legibility, and a per-mission / per-route Log window round it out.
- **Mod support** - Waterfall (with config packs such as Stock Waterfall Effects) and ReStock / ReStock+ ghosts keep correct engine plumes and RCS puffs in every configuration.

v0.10.2 - Career-Safety and Render Cutover

- **Career-safety fixes** - fixed a career freeze on the first climb to space (a repeated world-record recalc storm), a discard-drops-the-contract-you-earned corruption, and the re-aimed heliocentric transfer rendering tilted out of plane.
- **Map render cutover cleanup** - the old per-frame orbit-line show / hide code was removed now that the modular render pipeline owns those decisions, so a driven ghost's line, icon, and visibility come from one source.
- **Code health** - a behavior-neutral refactor pass split large methods and de-duplicated shared route / settings / codec logic across the systems added since the last cleanup.

v0.10.3 - Logistics Feature-Complete and Multi-Moon Missions

- **Inter-body Supply Routes (M5)** - Supply Routes now run between planets: delivery follows the transfer windows the ghost flies, and the dispatch cadence delivers every Nth window. With this, logistics milestones M1-M6 are all shipped.
- **Logistics legibility complete (M6)** - route paths draw on the map and Tracking Station, held routes name their exact blocker in the Status cell (including escrow contention naming the reserving route), the detail panel lists recent per-cycle cargo flow, candidates can be dismissed, and committing a qualifying flight prompts you to create a route. Undock-to-undock depot shuttles are now routable.
- **Claw producer** - supply runs that transfer across the Advanced Grabbing Unit (including asteroid grabs) can become routes, the second connection producer after docking.
- **Multi-moon and dual-constraint missions** - a looped multi-moon tour (the Jool-5 profile) aligns its moons' joint configuration on replay; a mission that both lands and docks with a station at the same destination aligns both; and a mission docked by a vessel from another mission can loop its whole shared journey from its own side (Partner journey). Shapes that cannot align fail closed to a faithful replay with an amber reason.

v0.10.4 - Career Integrity, Missions and Basic Mode

- **Career books that add up** - long careers no longer silently lose accepted contracts (Parsek recorded the time allowed instead of the due date; old saves are repaired on load and the lost contracts restored), a researched tech node can no longer be quietly re-locked, and money, science, reputation and crew experience earned before or during a re-flight survive the merge. Strategy payouts, skims and currency trades are recorded, and recovery refunds and crew experience are filed against the flight they belong to.
- **No more lost missions or vessels** - a quickload can no longer cost a mission (neither a vanished mission nor a ghost-only commit of an interrupted flight), debris outcomes survive leaving a flight, discarding a flight cleans up its stranded recording files, and rewinding to re-fly a separation no longer removes every other real vessel from the game.
- **Missions and Basic mode** - the Recordings window becomes Missions, with a plain-language summary, a readable event history, chapters for sub-stories and named dock partners per mission. A Basic mode shows only Timeline, Missions, Logistics and Settings; Advanced is the full UI.
- **Leaner settings, auto-merge on** - nine settings were removed or hardwired, and auto-merge ships ON, so leaving a flight commits the mission (a Re-Fly exit and quitting to the main menu still ask). The one-time pre-Parsek save backup is now unconditional and verifies its own work.
- **Supply Routes, Watch mode and ghosts** - routes run between planets on real transfer windows, draw their path on the map and Tracking Station, name their exact blocker, wait instead of losing cargo when a destination is full, and survive a rewind. Watch mode works within its promised 300 km and no longer runs on with nothing to watch when a loop starts its next lap.

Ghosts animate plumes, panels, gear, bays, drills and EVA jetpacks over their real durations, and a disabled control's hover says what is blocking it.

v0.10.5 - UI Refresh

- **A full UI refresh** - Real Spawn Control, Logistics (with a new Route History window replacing the old route logs), Missions, the Mission Log, Kerbals, the Timeline (with a new Career view), the Recordings tab and Settings are redesigned or rebuilt under one shared window style: the same countdowns, colours, "Go to" and Interact buttons, empty-list wording and column alignment everywhere. The Career window is removed (its counts live in the Timeline), as is every recording-deletion and data-wipe surface. KSP's own KSC screens - R&D, Mission Control, Administration, the Astronaut Complex, the VAB/SPH crew dialog, part purchases and the facility menus - grey out actions the committed timeline has reserved, with a short dated reason on hover, using stock's own controls instead of Parsek badges.
- **Career and KSC reservations** - research, contract accept / decline / cancel, facility upgrades and repairs, hires and dismissals, strategy activation, part purchases and kerbal EVA or transfer are refused with a reason when the committed timeline needs them later; KSC building destructions and repairs join the career history, and the ledger follows non-Normal difficulty settings.
- **Re-Fly and rewind correctness** - Re-Fly is limited to vessel separations (an EVA kerbal is never an Unfinished Flight; old EVA Rewind Points are removed on load), a Re-Fly survives dropping a stage or switching away, and re-flies with an EVA or an undock replace the old flight on merge. Rewinds no longer delete an earlier launch of the same craft or a vessel an earlier flight left behind, nor spawn a second copy of a landed vessel, and a Rewind-to-Launch taken mid-Re-Fly ends that Re-Fly.
- **Recordings and replay** - recordings can no longer be deleted (Archive hides one instead); recording fixes for long frames, doubled staging samples, optimizer-split flights and PersistentRotation spin; a ground part placed on EVA is its own vessel that replays and spawns whole after a rewind; recovered vessels are saved as Recovered and credited to their own flight by identity; ghosts count for CommNet as the vessel they replay; and Parsek says on screen when it starts up with a failed patch or from the wrong folder.
- **Logistics updates** - routes are named after the mission they repeat, the Logistics window and Route History are rebuilt, a held route says how much fuel is missing, loading a save no longer stops routes, and cargo really moves between nearby vessels during time warp.
- **Automated test harness buildout** - the harness grows to 367 committed scenarios with every registry coverage cell claimed (253 of 253, `docs/dev/autotest-status.md`), every in-game test group driven by an unattended run, a game-settings axis, physics-warp recording lanes, GUI census lanes that photograph Parsek's windows and the stock-screen annotations, and mutation checks that show a lane's gates can fail.

Phase 13: Logistics (Supply Routes) - shipped in v0.10.0

Stock-first automated cargo delivery from proven player-flown Supply Runs. Shipped in 0.10.0; summarized under Completed (v0.10) above. Full design: `docs/parsek-logistics-supply-routes-design.md`. Fly a cargo run once, dock, transfer stock cargo, undock, commit, then create a recurring Supply Route from the eligible Candidate in the Logistics window.

- **v0 route shape**: docking-only, delivery-only, single-stop Supply Routes created from complete dock/transfer/undock Supply Runs. Claw/grapple/crossfeed, pickup routes, crew delivery, multi-stop routes, and round-trip linking are deferred (future work).
- **Stock proof-of-work** — route creation derives the delivery manifest from connection-scoped snapshots: cargo must leave the transport-side part set and appear on the endpoint-side part set during the docking window. Aggregate merged-vessel totals are not enough proof.
- **Origin cost model** (owner ruling 2026-10-21: being removed; supply routes will carry no money and Space Center routes become free in every mode, because recorded route money and free interaction with route vessels cannot both hold; logistics design section 10.6.1): KSC-origin Career routes charge a stock-realistic funds cost for the source vessel parts plus used/delivered cargo, and credit the recovered transport cost back one cycle later. Only KSC-origin routes dispatch in v0. A non-KSC (vessel) origin route can be created (its start-docked origin proof is captured), but the per-cycle origin-cargo debit is intentionally stubbed, so the route holds in WaitingForResources and never dispatches; non-KSC origin support is post-v0.
- **UT-driven scheduler** — route progress, delivery, pause behavior, save/load catch-up, and time-warp behavior are driven by universal time in `ParsekScenario`, with ghost playback treated as visual evidence rather than authoritative timing.
- **Endpoint resolution** — orbital endpoints use recorded vessel PID only. Surface endpoints prefer the recorded PID and may fall back to one nearest compatible stock vessel near the recorded coordinates, never to an abstract area warehouse.
- **Visual presence**: ghost supply vessels replay the recorded run as a looped mission segment, but route execution is pure math: deduct at origin, wait, add to destination. No physical vessel is spawned during transit.

Logistics prerequisites delivered for Phase 13: Phase 11 provides base vessel-level resource and inventory manifests. Supply Routes also required connection-scoped capture extensions: dock/undock resource and inventory manifests by transport/endpoint

part PID set, `TransferTargetVesselPid`, `TransferKind`, `TransferEndpointSituation`, `StartDockedOriginVesselPid`, and exact `InventoryPayloadItem` snapshots from canonical `STOREDPART` data so inventory deliveries preserve per-item state.

Every supply ship remains a replay of a real mission the player flew, but v0 deliberately keeps the mechanics narrow so the first implementation is reliable and stock-realistic.

Render-confirmation coverage (testing direction). The network can run any route the player can fly, but automated confirmation that a looped route RENDERS accurately in map view, the Tracking Station and the KSC scene covers only part of the origin → destination space. The class taxonomy, gap register and ranked plan to close it live in `docs/dev/autotest-roadmap.md` → "The loop-render coverage program".

Logistics: milestones to feature-complete (M1-M6, all SHIPPED)

Status: COMPLETE as of 0.10.3. M1 through M6 are all shipped, and the claw was lifted in as the second connection producer, so Supply Routes now cover any route the player can fly. The milestone sequence below is retained as the record of how it was built, each item marked with its shipping release.

The path from the shipped v0 to feature-complete is owned by **section 19 of `docs/parsek-logistics-supply-routes-design.md`** — the provenance model (provenance → transfer → disposition, with flow closure as the single validity rule) and the milestone sequence below. It replaces the earlier Tier 1-4 list (each old tier item is mapped to its milestone in section 19.4); the per-item system analysis stays in design doc section 17.1. Milestones are ordered by gameplay-per-effort, not committed versions; M6 runs in parallel with the others.

- **M1 — The network unlock: non-KSC origin debit. SHIPPED (0.10.2-dev, 2026-06-10).** Built `LiveOriginDebitWriters` + the physical `RouteCargoDebited` debit (a direct write at the loop dock crossing carrying the actual-debited manifest; `RouteModule` stays observe-only, the draft's "applier through RouteModule" wording was corrected in design doc 19.4), un-stubbed `LiveRouteRuntimeEnvironment.OriginHasCargo`, docked-start origins only with the origin location descriptor now captured for proximity rebuild. Pulled forward **dispatch priority** (persisted `Route.DispatchPriority` + deterministic tick comparator + detail-panel stepper) and the **scenario-lifecycle test harness** (3-layer: codec round-trips, ordered source-text gates, in-game lifecycle tests for the loaded/unloaded debit + save round-trip). Added the undocked-start workflow rejection (`RouteAnalysisStatus.UndockedStartOrigin`); the undocked-start origin PROVER is retired by the provenance doctrine (`docs/dev/logistics-origin-ownership-proposal.md` is superseded). Inventory-payload debit deferred to M3. Hub-and-spoke networks emerge here from composition alone.
- **M2 — Resource generality: mod resources + harvest provenance. SHIPPED (0.10.2-dev, 2026-06-12).** The transferability rule has one authority (`ResourceTransferability`: any defined resource routes, EC/IntakeAir excluded, undefined names excluded from admission-direction outputs and logged instead of priced at zero); CRP-style fixtures cover capture/analysis/delivery/funds; every active-stop recording carries a full-run transport-scoped cargo manifest (`RouteRunCargoManifest`, presence-gated so pre-M2 recordings analyze exactly as before); harvest windows capture `BaseConverter`-derived activity (stock and modded drills, asteroid/comet drills on an already-grappled asteroid, warp re-baselined at rails transitions); the analysis admits witnessed harvested gains and rejects untracked gains naming the exact quantity (`UntrackedCargoGain`); fully-harvested undocked runs become harvest-origin routes (empty cost manifest, no depot debit) and mixed runs reduce the depot debit by the witnessed harvest; the KSC funds charge for routes built from new recordings is launch-manifest based (existing routes keep their exact cost; design doc 19.2.4 decision). Stated non-goal: background production on unloaded vessels. Deferred: BG-leg harvest capture (run manifest voided, tree degrades to legacy analysis), drilling while docked (M3), N-window accounting (M4).
- **M3 — Direction generality: pickup, then mixed windows. SHIPPED (0.10.2-dev, 2026-06-15; M3a pickup direction, PR #1160).** The generalized transfer-direction model, paid for by pickup: source-side connection capture, availability probe, reverse-direction writers; `HasResourcePickup` classifies instead of rejecting; flow accounting replaces assume-start-loaded analysis; then per-window bidirectional manifests. Multi-origin semantics (all-or-nothing source gating + the RouteStore cargo-escrow reservation map) were resequenced to M4b (design doc 19.4).
- **M4 — Shape generality: multi-stop, then round-trip. SHIPPED (0.10.2-dev, 2026-06-20; multi-stop M4a + multi-origin M4b + round-trip M4c, PR #1180).** The multi-window model, paid for by multi-stop: N ordered windows in analysis, per-window delivery on the loop clock, one endpoint per stop (`Stops` / `DeliveryOffsetSeconds` already reserved in the save shape). Round-trip follows as a thin `LinkedRouteId` chain-constraint scheduler. Verified Missions-boundary scope: end-trims `[launch..dockN]` work with today's locked-layer boundaries; mid-recording start-trims (undock→undock shuttles) stay a documented limitation (Missions gap 1).
- **M5 - Reach: inter-body activation. SHIPPED (0.10.3, PR #1238; gate passed in-game 2026-07-08).** Routes dispatch on the same launch windows the ghost renders, with `DispatchInterval` N applied as a residual-cadence modulo on the window index (OFF for flat routes, 1 for zero-drift schedules, N for re-aim windows; the earlier "modulo on the scheduled-launch index" sketch was superseded by the ratified plan, see design doc 0.9/19.4). Route-layer only (the Missions / GhostPlaybackLogic / re-aim diff is empty), so flat routes stay byte-identical on disk. Completes "any route the player can fly, the network can run."

- **M6 — Legibility (parallel track). COMPLETE (2026-07-07).** Every non-dispatching route says why, in player language, at row level (live-route hold reasons + escrow-caused shorts naming the reserving route + the Status-cell row-level treatment, all SHIPPED; candidate near-misses already surface reasons); structure list window (SHIPPED 0.10.1 as the mission / route Log window); candidate intent helper (SHIPPED 0.10.3) + the Record Supply Run commit-time prompt (SHIPPED 0.10.3-dev, branch `logistics-m6-closeout`); map-view route lines (SHIPPED 0.10.3); per-cycle flow display (SHIPPED 0.10.3); precise per-run recovery landing CLOSED won't-build by maintainer decision (2026-07-07, per `docs/dev/research/logistics-recovery-clock-memo.md`; the constant deferred recovery credit is the ratified permanent behavior).

Out of scope by doctrine (revisit on demand): non-docking connection producers beyond the claw (stock crossfeed/fuel-line: continuous-flow, no couple event, no window to snapshot), crew delivery, persisting-transport materialization, undocked-start origin proving. Claw/grapple LIFTED and shipped 2026-07-07 (branch `logistics-claw-producer`): claw couples are classified and admitted as the second connection producer, including asteroid (PotatoRoid) grabs; see `docs/dev/design-logistics-claw-producer.md` and the decompile findings at `docs/dev/research/claw-grapple-coupling-internals.md`. Already resolved earlier: "Dispatch now" (subsumed by Send Once), dock-side-baseline edge case (shipped 0.10.1), structure list window (shipped 0.10.1 as the mission / route Log window: pure builders `MissionStructureListBuilder` / `RouteStructureListBuilder` feeding a reusable `StructureListWindowUI`, opened from the "Log" button on each Missions-tab mission header and the "Log (Route)" / "Log (Mission)" buttons in each Logistics route detail panel; the route Log was retired in 0.10.5, and a route's single "Log" button opens its source mission's Log).

Phase 14: Cooperative Async Multiplayer

Status: deferred (owner ruling 2026-10-24). Implementation does not start before Parsek's public release, and comes later only if Parsek has users. See "Current focus" above.

Multiple players run one shared space program: one KSC, one pooled economy (science, funds, reputation, kerbals; contracts stay local-only in v1), one timeline with per-player attribution. The Kerbal system feels populated with vessels flying and bases being built. Players never need to be online simultaneously; the timeline converges over time - correspondence chess, not real-time multiplayer.

Design (binding): `docs/dev/design-coop-async-multiplayer.md` (2026-09-01). It supersedes this section's earlier sketch in two ways:

- **No Gloops prerequisite.** The exchange uses Parsek-native full-fidelity packets (recording metadata + sidecars + ledger actions), because imported missions are full timeline citizens that feed the ledger, spawn real vessels, and support docking - all things a stripped `.gloop` deliberately cannot carry. The Gloops extraction (see below) remains on the roadmap for the standalone-mod goal but is no longer on this feature's critical path.
- **Full citizenship, not ghost-only import.** The earlier "purely additive ghosts, no conflict with live game state" model is replaced by: per-player append-only contribution folders in a file-sharing-synced shared folder, foreign recordings imported as read-only citizens, real spawns of peers' terminal vessels with dock/board interaction, a deterministic arbitration fold for concurrent-interaction conflicts, and a merged economy (deterministic walk order, once-ever spend dedup, debt semantics, earlier-UT milestone credit).

Carried forward from the original sketch: the shared cloud-synced folder, player identity + attribution, and the correspondence-chess framing. Launch pad allocation is not part of v1.

Gloops Extraction (decoupled from Phase 14)

Revised 2026-10-05 (owner rulings; full design in `docs/dev/gloops-recorder-design.md`). Gloops becomes a standalone mod in its own repository; the in-Parsek Gloops recorder is deleted; there is no runtime API and no control of a standalone install, and the two record separately. No longer a Phase 14 prerequisite (see above). No extraction is scheduled (design section 12, 2026-10-05): Parsek is the priority, and a future standalone Gloops is expected to fork Parsek's ghost code at that moment rather than share a maintained Core (the shared-Core design, a Core compiled into `Parsek.dll` from a submodule, is kept in the design doc as a reference only). The in-Parsek Gloops recorder is deleted (PR #2022); removing player-authored looping (`docs/dev/plans/remove-player-looping.md`) is on hold (owner, 2026-10-13).

Extraction scope (shared-Core path, reference only):

- Core stays in this repository (top-level `Gloops/Core/`, compiled by `Parsek.csproj`) until engine churn settles; the submodule split is the last step
- Leaf moves first (data structs, FX, zones, audio, config, logging); then invert the engine's direct reads of `RecordingStore` / `ParsekFlight` / re-fly state into host interfaces and split `GhostPlaybackLogic` (engine vs. policy)

- One per-vessel part-event poller and sampler replaces the duplicated foreground / background wrappers
- Standalone take recorder: the original vessel and its children only, ending on any interaction with an external vessel

Standalone Gloops mod (if demand exists):

- Split into separate repository / submodule
- Content pack system for ambient world activity (KSC traffic, scenery)
- Standalone UI (loop manager, pack toggles, settings)
- Custom mesh support for non-vessel content

Phase 15: Competitive Play & Space Race

Player boundaries that enable competitive multiplayer. Each player has their own game actions timeline (separate science, funds, reputation, contracts, kerbals). Only things with visible effect on the common game world are shared — vessel recordings play as ghosts in everyone's game, but each player's economy is independent.

Player Boundaries

- **Per-player game actions timeline** — each player's resource ledger is independent; importing another player's recordings adds their ghosts to the world but does not affect the local player's science, funds, or reputation
- **Shared world layer** — vessel recordings (trajectories, part events, ghost chains) are shared and visible to all players; the physical world is common, the economic world is separate
- **Milestone visibility** — all players can see each other's milestones (first orbit, first Mun landing) but milestones only feed into the achieving player's resource pipeline

Space Race

- **Pre-built recording packs** — a set of recordings representing an AI space program's progression (first orbit on Day 30, Mun landing on Day 150, etc.); hand-crafted or community-contributed
- **Milestone racing** — Parsek already tracks milestones; space race is a comparison UI showing both programs' achievements side by side
- **Difficulty scaling** — different packs for different skill levels
- **Community races** — players export their full campaign as a space race opponent ("Can you beat Scott Manley's career run?")

The Dependency Chain

```
v0.3–v0.5: Core Recording System (COMPLETE)
    |  Recording, playback, loops, ghost chains, spawn safety,
    |  time jump, rendering zones, multi-vessel sessions
    |
    ▼
v0.6: Game Actions System (COMPLETE)
    |  Science, funds, reputation, kerbals, contracts,
    |  facilities, strategies, reservations, recalculation
    |
    ▼
Phase 9: Timeline (v0.7 ✓)
    |  Unified chronological view of all committed events,
    |  significance tiers, filtering, rewind from timeline
    |
    ▼
Phase 10: Location Context (v0.7 ✓)
    |  Recordings know WHERE they start/end (body, biome, situation)
    |
    ▼
Phase 11: Resource Snapshots (done)
    |  Recordings know WHAT they carry
    |
    ▼
Phase 11.5: Recording Optimization & Observability (v0.8.x)
    |  Observability + ghost LOD + trajectory/snapshot shrink shipped;
```

```
     |  remaining follow-up is synthetic stress benchmarking/tuning
     |
     ▼
Phase 12: Rewind to Separation (v0.9 ✓)
     |  Rewind Points at multi-controllable splits, Unfinished Flights
     |  group, append-only supersede, broad reviewed-career tombstones.
     |  Independent of
     |  Phase 13 — both consume Phase 11 resource/inventory/crew manifests.
     |
     ▼
Phase 12.5: Stable Leaves + Re-Fly Hardening (v0.9.1 ✓)
     |  STASH, Fly/Seal/Stash, slot-aware stable leaves, linear supersede,
     |  quickload/visibility/relative-playback hardening
     |
     ▼
Phase 13: Logistics (Supply Routes) (v0.10.0 ✓)
     |  Stock-first Supply Routes from proven dock/transfer/undock Supply Runs
     |  (v0 cut: docking/delivery/single-stop/same-body; pickup, multi-stop, round-trip,
     |   non-KSC origins, claw and inter-body routes followed in v0.10.1-v0.10.3, M1-M6)
     |
     ▼
Phase 14: Cooperative Async Multiplayer (deferred until after the public release)
     |  Parsek-native packet exchange, shared folder, campaigns,
     |  player identity, pooled economy, arbitration fold
     |  (design: docs/dev/design-coop-async-multiplayer.md;
     |   Gloops extraction decoupled - no longer a prerequisite)
     |
     ▼
Phase 15: Competitive Play + Space Race
     |  Per-player game actions, milestone racing, opponent packs
     |
     ▼
Phase 16: Mod Compatibility
     |  Kerbal Konstructs, KSC Switcher, Extraplanetary Launchpads,
     |  off-world construction, modded launch sites via reflection
     |
    ...
Standalone Gloops Mod (if demand exists)
       Separate repo, content packs, standalone UI, custom meshes
```

Phase 16: Mod Compatibility

All prior phases target stock KSP (including Making History DLC). This phase adds support for popular mods via reflection-based detection and API integration. Only attempted after the stock experience is stable and fun.

- **Kerbal Konstructs / KSC Switcher** — detect custom launch sites, capture site name at recording start, display in timeline and Recordings Manager
- **Extraplanetary Launchpads** — detect EL-spawned vessels (builder PartModules: `ELLaunchpad`, `ELDisposablePad`, `ELSurveyStation`), capture pad name, handle empty `launchedFrom` field
- **Off-world construction mods** — detect vessels built outside KSC, tag recordings with construction origin
- **Mod detection pattern** — `AssemblyLoader.loadedAssemblies` (already used for PersistentRotation, RemoteTech). Reflection for API access. Graceful degradation when mods are absent.
- **Compatibility testing** — CustomBarnKit, Strategia, Contract Configurator (see T43)

Deferred

Planetarium.right drift compensation for long orbital segments

KSP's inertial reference frame may drift over very long time warp durations. This could cause ghost orientation mismatch for interplanetary transfer segments. Needs empirical measurement first — if drift is sub-degree for typical segment lengths, no fix needed.

Crash-safe pending recording recovery (partial)

Commit crash window closed (sidecar files flushed immediately). Remaining gap: stashed-but-not-yet-committed recordings (merge dialog pending) still live only in RAM. Solution: write a pending manifest file when a recording is stashed, auto-recover on next load if the recording wasn't committed or discarded. Low priority — remaining gap is stash→commit only.

Additional part event coverage

- Two-phase engine startup (spool-up animations on some engines)
- Engine plate covers / interstage fairings (partially fixed — mesh cloned but positioning may be wrong)

Ghost LOD follow-up

Distance-based ghost LOD shipped in `0.8.1`, including the hidden-tier ghost unload/rebuild follow-up. Particle pooling for engine/RCS FX is not scheduled; the Phase 11.5 outcome there was observability/measurement only. Storage-side optimization is also now shipped, so the remaining Phase 11.5 follow-up is synthetic stress benchmarking/tuning.

~~Kerbal reservation refactor (T44)~~ - done

Shipped in 0.6.2: reserved kerbals keep their natural roster status and a Harmony patch handles the VAB/SPH crew dialog, which removed both workaround patches. In 0.10.5 the dialog shows reserved kerbals greyed out with the reason instead of hiding them.

Hyperbolic escape orbit line rendering (189b)

Ghost escape orbits clip at finite distance (~12,000 km). Active vessels show full escape trajectory via PatchedConicSolver which ghosts don't have. Needs custom LineRenderer or orbit line extension.

Out of Scope

- **Racing modes or lap timing**
- **AI playback or autopilot**
- **Real-time multiplayer synchronization** — Parsek's multiplayer model is async (Phases 13–14). No shared physics simulation, no lockstep networking.
- **Taking control of recorded vessels mid-playback** — jumping into a live ghost while it is playing out creates unresolvable paradoxes (recording future events, reserved crew, applied resource deltas). The complexity is not worth the payoff. **Note:** the narrower "Rewind to Separation" feature (see above) does allow re-flying a sibling vessel from a past split event — that works because it rewinds UT to the split moment and replaces the sibling's BG-crash via append-only supersede, rather than hijacking an in-flight ghost.
- **Timeline branching or alternate histories**